

一、在 **gitee**、**github** 或 **gitlab** 中选择加入一个开源项目，说明你所做的工作以及体会。

项目：OpenMoji 是一个开源项目，旨在创建一个免费、可定制的表情符号库。

所做的工作：

1. 项目研究：阅读 OpenMoji 的 README 文件和文档，了解项目的目标和用途。查看项目的代码库，理解代码结构和开发流程。

面向设计师、开发人员和其他所有人的开源表情符号！OpenMoji 是 HfG Schwäbisch Gmünd 的一个开源项目，由 Benedikt Groß、Daniel Utz、70+ 学生和外部贡献者完成。

👉 [OpenMoji.org/](https://openmoji.org/)

交互、创建、保存和共享您的工作！ #openmoji

此 GitHub 存储库包含 OpenMoji 项目的所有源文件和导出的 png/svg 文件。

⚠️ 请注意，master 分支正在积极开发中，因此如果您正在寻找稳定的生产版本，请使用其中一个版本。

🔗 您可以通过 [OpenMoji Dev Catalog](#) 查看最新的开发工作，该目录列出了 master 分支的所有 OpenMoji。

## 目录

- [风格指南](#)我们心爱的风格指南。
- [常见问题](#)检查您的问题是否已得到解答
- [贡献](#)欢迎 Pull Requests!
- [开发人员设置](#)如何设置环境。
- npm OpenMoji 包的 [API](#) 文档。
- OpenMoji-Color 和 OpenMoji-Black 字体的[字体](#)信息。
- 所有作者和贡献者的[团队](#)列表。
- [确认](#)谢谢!
- [行为准则](#)和 OpenMoji 社区声明。

2. 设置开发环境：按照项目的 CONTRIBUTING 指南，设置本地开发环境。克隆代码库（`git clone https://github.com/your-username/openmoji.git`），并确保能够成功运行项目的测试。

## 开发人员设置

### Node.js

1. 安装 [node.js](#) (请参阅文件 `.nvmrc` 中的版本) )
2. 打开终端并导航到您下载到计算机上的文件夹: `openmoji`

```
cd path/to/folder
```



3. 跑:

```
npm install
```



### bash 脚本

如果您想运行文件夹中的 bash 脚本 (.sh 文件) , 例如用于导出 png 文件, 则必须[安装额外的依赖项](#)。

`helpers`

`npm test` 运行测试, 测试成功, 开发环境部署完毕。

OpenMoji **15.0.0**

# Open source emojis for designers, developers and everyone else!



Search emoji ...

3. 贡献代码: 根据项目的编码规范编写代码, 并提交 Pull Request。

- [🚀 通过 Github 和 Pull Requests 贡献一个 Emoji \(首选\)](#)

- [文件名和文件夹](#)
- [设计你的表情符号](#)
- [规范化 SVG 格式](#)
- [测试您的设计](#)
- [关于 The Emoji 的信息](#)
- [提交](#)
  - [如何提交 Pull Request](#)

确保本地仓库与原始 OpenMoji 仓库同步，添加上游仓：

```
git remote add upstream https://github.com/hfg-gmuend/openmoji.git
```

从上游仓库拉取最新的更改并合并到你的本地仓库：

```
git fetch upstream
```

```
git checkout master
```

```
git merge upstream/master
```

修改相关内容并完成推送操作：

```
git add .
```

```
git commit -m "first update"
```

```
git push origin your-branch-name
```

创建 Pull Request：

点击“Pull Request”按钮，选择你的分支作为比较对象，点击“Create pull request”。

## 比较更改

在下面比较跨分支、提交、标签等的更改。如果需要，您还可以[跨分支比较](#)。

 基础：主人 ▾ < 比较：主人 ▾

选择上面的不同分支或分支来讨论和审查更改。[了解拉取请求](#) [创建拉取请求](#)

填写 PR 的标题和描述，解释你所做的更改和原因。

点击“Create pull request”提交你的 PR。

4. 参与讨论：在我的 Pull Request 中回应审查者的评论，并根据反馈进行代码修改，并在 Pull Request 中说明修改的内容。在其他 Issues 和 Pull Requests 中提供帮助和反馈，维护良好的社区氛围。

# ESM 模块 #518



danchitnis 打开了这个问题 on Oct 7 · 7 评论



danchitnis 评论 on Oct 7

...

您好，感谢您的出色工作。有没有计划将 openmoji 的 [npm 模块](#) 迁移到 ESM? 似乎它仍在使用旧式的 commonJS 和 require。



b-g 评论 on Oct 7

成员

...

恐怕目前还没有计划。我们根本没有资源来使所有内容保持最新状态。跟上表情符号已经很困难了。不好意思！



b-g 于 on Oct 7 日结束此操作

5. 代码审查：审查其他贡献者的代码，理解他们所做的更改，检查代码是否遵循项目的编码规范和风格。在评论中提供具体的改进建议，而不是仅仅说“我不喜欢这个”。
6. 文档编写和维护：发现文档中有不清晰的地方，提出改进，并更新文档。
7. 解决 bug：在测试新表情符号时发现一个显示问题，并修复它。
8. 功能开发：开发一个新功能，比如允许用户自定义表情符号的颜色。

体会：

加入 GitHub 上的 OpenMoji 项目，我深刻体会到了开源社区的技术深度和协作精神。在这个项目中，我不仅锻炼了我的编程技能，特别是在 SVG 图像处理和前端开发方面，还学会了如何高效地使用 Git 和 GitHub 进行版本控制和代码管理。通过阅读和理解项目文档，我学会了如何快速上手一个新项目，并根据项目的编码规范编写代码。在提交 Pull Request 后，我体验到了代码审查的过程，这不仅帮助我提高了代码质量，也让我学会了如何提供和接受建设性的反馈。此外，我还了解了项目的文档编写和维护工作，这让我意识到了文档对于项目的重要性，以及如何清晰地传达技术信息。总的来说，OpenMoji 项目不仅提升了我的技术能力，也加深了我对开源文化和社区协作的理解，这是我技术生涯中宝贵的财富。

二、加入课程案例项目 linuxer 的校内开发网站 (<http://192.168.217.240/linuxcourse/linuxer>)，对比 linuxer 和作业 1) 中的项目，结合 linuxer 使用过程中的问题，分析对 linuxer 项目需要进行的改进（每

组至少 3 个改进点，分析写在作业中），并提交到开发网站 **linuxer** 项目的 **issues** 中，命名为“**xx 时间 xx 小组：改进点名称**”，设置该 **issue** 的 **Milestone** 为“**实验 8 开源项目开发管理**”

## 1、添加社交功能

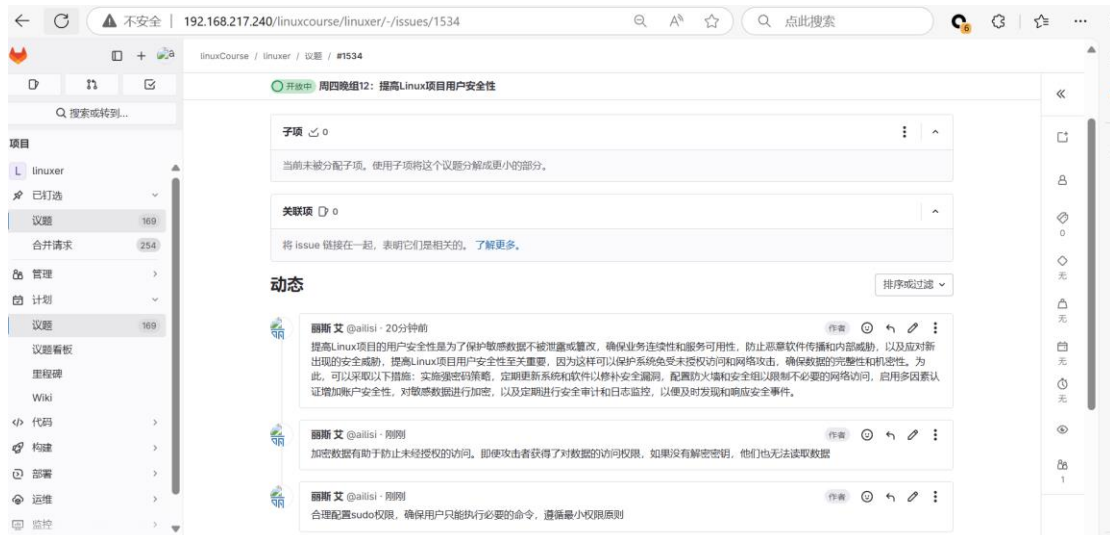


现有功能中，我们的社交功能的实施性几乎为零，所以我们准备添加一个社交功能以提供一个用户交流的平台来将用户连接起来

改进点：

- (1) 集成现有的社交网络平台：可以通过集成如 **SocialSdkLibrary** 这样的库来实现微博、微信、QQ 等社交平台的登录和分享功能，这样可以让用户使用社交账号登录项目，并分享内容到社交网络。
- (2) 开发专门的社交流：可以开发一个基于 **Linux** 的聊天工具，类似于在 **CSDN** 博客中提到的聊天功能设计与实现，使用 **UDP** 协议进行网络通信，实现用户之间的实时消息传递。
- (3) 使用开源社交网络平台：可以考虑使用如 **DIASPORA\***、**Friendica** 或 **Elgg** 这样的开源社交网络代码，这些平台提供了基本的社交功能，并且可以自定义和扩展以适应项目的需求。

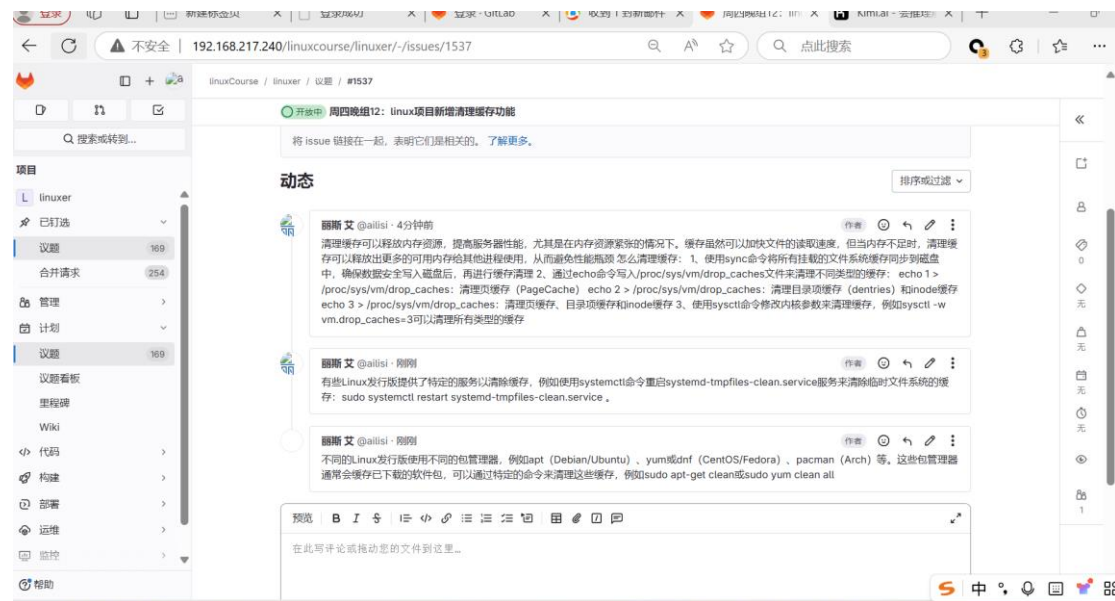
## 2、提高 linux 项目用户安全性



作为一名信息安全专业的学生，我发现我们的项目中存在安全性过低的问题，针对这个问题，我采取了以下方案进行改进：

- (1) 定期更新系统：更新系统可以修补系统中的漏洞，并保证系统的安全性。
- (2) 使用强密码：使用难以猜测的强密码非常重要，强密码应至少为 12 个字符，包含大小写字母、数字和符号的组合。
- (3) 禁用不必要的服务：某些服务可能不需要，并且可能会产生潜在的安全风险。禁用这些服务可以减少攻击面。
- (4) 安装和使用防火墙：防火墙可以拦截来自外部网络的攻击，并在系统中监控数据流量。
- (5) 加密数据：加密数据有助于防止未经授权的访问。即使攻击者获得了对数据的访问权限，如果没有解密密钥，他们也无法读取数据。
- (6) 使用防病毒软件：安装防病毒软件可以帮助检测和清除恶意软件，保护系统不受病毒和恶意软件的侵害。
- (7) 限制 root 账户的使用：避免使用 root 账户直接登录系统，而应使用非 root 账户登录，并在需要时使用 sudo 命令获取必要的权限。
- (8) 配置 sudo 权限：合理配置 sudo 权限，确保用户只能执行必要的命令，遵循最小权限原则。
- (9) 监控系统日志：定期检查系统日志可以帮助发现潜在的安全问题。日志文件通常存储在 /var/log 目录下。

### 3、linux 项目新增清理缓存功能



缓存产生于文件系统访问、数据库查询、网络请求和应用程序运行等多个环节。当文件被读取或写入时，系统会将数据暂存于内存中的页缓存以加速后续访问。数据库和网络应用也会缓存查询结果和资源以提升性能。此外，系统还会缓存目录项和 inode 信息、编译结果以及图形渲染数据。这些缓存虽然提高了效率，但也需定期清理以释放资源。

改进方案：

- (1) 使用 GUI 工具清理缓存：有些 Linux 发行版提供了图形界面的工具，如 BleachBit、Stacer、Ubuntu Cleaner 等，这些工具可以帮助用户直观地选择和清理缓存文件。
- (2) 清理包管理器缓存：不同的 Linux 发行版使用不同的包管理器，例如 apt (Debian/Ubuntu)、yum 或 dnf (CentOS/Fedora)、pacman (Arch) 等。这些包管理器通常会缓存已下载的软件包，可以通过特定的命令来清理这些缓存，例如 `sudo apt-get clean` 或 `sudo yum clean all`。
- (3) 清理特定服务的缓存：有些 Linux 发行版提供了特定的服务以清除缓存，例如使用 `systemctl` 命令重启 `systemd-tmpfiles-clean.service` 服务来清除临时文件系统的缓存：`sudo systemctl restart systemd-tmpfiles-clean.service`。
- (4) 清理数据库缓存：如果项目使用了数据库，数据库服务通常会有自己的缓存机制，可以通过重启数据库服务或者使用相应的命令来清除数据库缓存。
- (5) 重启应用程序或服务：如果以上方法无效或不方便，可以考虑重启应用程序或服务来清除缓存。重启应用程序将清除当前运行的进程及其相应的缓存。

